Blink “A Ciencia por Tras da Busca Vetorial”

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Blink "A Ciência por Trás da Busca Vetorial"

Alberto Souza

2026-02-23

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Contents

O que você vai aprender com este curso? i

Dense embeddings comprimem significado em um único vetor – e isso tem um preço 1

Similaridade e relevância não são a mesma coisa – e confundir as duas é o erro mais comum 3

Sparse vectors capturam o que os dense embeddings não conseguem: a correspondência exata de termos 5

Hybrid search une keyword matching e busca semântica em um único pipeline 7

Reciprocal Rank Fusion resolve o problema de combinar rankings de escalas diferentes 9

Re-ranking com ColBERT refina resultados usando interação token a token 11

O Qdrant suporta múltiplos tipos de vetores no mesmo documento, e isso muda tudo 13

Semantic chunking agrupa parágrafos por significado, não por posição no texto 15

HDBSCAN encontra clusters automaticamente sem você precisar definir quantos existem 17

Metadados estruturados transformam seu banco vetorial em uma base de conhecimento filtrável 19

Resumo Final 21

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Sobre este resumo 23

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

O que você vai aprender com este curso?

Você já se perguntou por que, ao digitar uma pergunta em um sistema de busca, os resultados parecem "entender" o que você quis dizer – mesmo quando você não usou as palavras exatas que estão nos documentos? E já percebeu que, às vezes, o resultado mais "parecido" não é o mais útil? A diferença entre **similaridade** e **relevância** é uma das questões centrais da busca vetorial, e dominar essa diferença é o que separa sistemas medíocres de sistemas realmente eficazes.

Neste curso, Daniel Romero mergulha fundo na ciência por trás da busca vetorial, construindo passo a passo um sistema de retrieval para documentos financeiros da SEC americana. Mais do que teoria, o curso entrega um pipeline completo e funcional – de ingestão de dados até re-ranking com modelos neurais.

Ao longo destes blinks, você vai descobrir:

- Por que **dense embeddings** são poderosos mas insuficientes quando usados sozinhos – e como um score de 60% de similaridade pode ser decepcionante.
- Como **sparse vectors** e **BM25** capturam correspondências exatas que os embeddings semânticos perdem.
- Como **hybrid search** combina o melhor dos dois mundos para cobrir mais cenários de busca.
- Por que o **Reciprocal Rank Fusion (RRF)** é a forma elegante de unificar rankings que vivem em escalas completamente diferentes.
- Como o **ColBERT** usa interação token a token para re-ranquear resultados com precisão cirúrgica.
- Por que **semantic chunking** com HDBSCAN produz chunks

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

mais inteligentes do que a simples divisão por parágrafos.

- Como **metadados** estruturados transformam um banco vetorial em uma base de conhecimento filtrável.
- E por que o pipeline ideal segue uma hierarquia: velocidade primeiro, precisão depois.

Se você trabalha com IA, RAG ou sistemas de busca e quer entender como construir retrieval de alta qualidade do zero, este resumo vai te dar a base técnica e prática para isso.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Dense embeddings comprimem significado em um único vetor – e isso tem um preço

O ponto de partida de qualquer sistema de busca vetorial são os dense embeddings. Modelos como o all-MiniLM-L6-v2 pegam um trecho de texto e o transformam em um vetor denso de 384 dimensões, onde cada dimensão captura algum aspecto semântico aprendido durante o treinamento. A palavra "denso" significa que praticamente todos os valores do vetor são diferentes de zero – cada posição carrega informação.

Essa representação é poderosa porque permite comparar textos por significado, não apenas por palavras. Se você busca "riscos financeiros", o modelo entende que termos como "incertezas", "desafios" ou "mercado volátil" são semanticamente próximos. A busca é feita calculando a cosine similarity entre o vetor da query e os vetores dos documentos armazenados no banco vetorial.

Mas há um problema fundamental: ao comprimir todo o significado de um documento em um único vetor, nuances se perdem. Na prática, o curso demonstra isso com um pipeline básico de ingestão – ler um documento, dividir em chunks por parágrafos, gerar embeddings e inserir no Qdrant. O melhor score de similaridade obtido foi apenas 60%. Não é ruim, mas está longe de ser suficiente para um sistema de produção.

> *"This is the most basic ingestion pipeline. You read a document, split it into chunks, generate dense embeddings, insert them into Qdrant and can perform semantic searches. The best returned chunk had a similarity score of 60%. But what now? How can we improve this score?"*

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Embeddings densos são ótimos para capturar contexto semântico, mas nunca devem ser a única estratégia de busca. Se o seu sistema depende exclusivamente de dense vectors, você está deixando relevância na mesa. O score de 60% é um sinal claro de que há espaço para melhorar.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Similaridade e relevância não são a mesma coisa – e confundir as duas é o erro mais comum

Esse é um dos insights mais importantes do curso e uma armadilha em que muitos desenvolvedores caem ao construir sistemas de RAG. Um documento pode ser semanticamente similar à sua query sem ser relevante para o que você precisa. Imagine dois textos sobre o mercado financeiro que são muito parecidos entre si. Se você está buscando especificamente por riscos regulatórios da Apple, o documento mais similar pode falar sobre riscos da Microsoft – semanticamente próximo, mas completamente irrelevante para a sua necessidade.

O problema se agrava quando a query contém termos específicos que deveriam ser correspondidos exatamente. Se alguém busca por um ticker como "AAPL" ou um termo técnico financeiro preciso, a busca semântica pura pode não priorizar documentos que contenham exatamente esse termo. O modelo de embedding pode até entender que "AAPL" está relacionado a "Apple", mas não necessariamente vai dar o peso adequado à correspondência exata.

Essa distinção entre similaridade e relevância é o que motiva toda a evolução do pipeline ao longo do curso. Cada técnica adicionada – sparse vectors, hybrid search, re-ranking – existe para fechar essa lacuna entre "o que se parece" e "o que realmente importa" para a query do usuário.

> *"It's important to understand that similarity and relevance are not the same thing. A document can be semantically similar to your query, but not necessarily be the most relevant for what you need."*

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Ao avaliar seu sistema de busca, nunca se contente apenas com scores altos de similaridade. Pergunte-se: os resultados retornados realmente respondem à pergunta do usuário? Testes com queries específicas contendo termos exatos são a melhor forma de expor as fraquezas de uma busca puramente semântica.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Sparse vectors capturam o que os dense embeddings não conseguem: a correspondência exata de termos

Se dense embeddings são sobre significado, sparse vectors são sobre presença. Eles são gerados por algoritmos como o BM25, uma evolução do TF-IDF que calcula a relevância de um documento com base na frequência dos termos da query no documento, penalizando termos muito comuns e normalizando pelo tamanho do documento.

O resultado é um vetor onde a imensa maioria dos valores é zero – apenas algumas posições têm valores diferentes de zero, correspondendo aos termos que realmente aparecem no texto. Daí o nome "sparse" (esparso). Em vez de armazenar um vetor enorme cheio de zeros, o sistema armazena apenas os índices das posições com valores não-zero e seus respectivos valores. Isso economiza espaço e torna a busca muito mais rápida.

Na prática, sparse vectors brilham em cenários onde a correspondência exata de palavras é crucial. Em uma query como "what are Apple's financial risks in 2024?", o sparse vector vai fazer um match exato em "Apple" e "2024" se esses termos aparecerem no documento. Enquanto isso, o dense vector vai entender que "financial risks" está semanticamente relacionado a "uncertainties" ou "volatile market". Cada abordagem cobre uma lacuna que a outra deixa.

> *"BM25 calculates the relevance of a document based on the frequency of the query terms in the document, penalizing very common terms and normalizing by the document length. The result is a vector where most of the values are zero, with only a few non-zero values corresponding to the terms that appear."*

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Não subestime o poder dos sparse vectors. Em muitos cenários de produção, especialmente quando queries contêm nomes próprios, códigos, datas ou termos técnicos exatos, o BM25 vai encontrar resultados que os embeddings semânticos simplesmente perdem. A combinação dos dois é quase sempre superior a qualquer um isoladamente.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Hybrid search une keyword matching e busca semântica em um único pipeline

Hybrid search é a técnica que combina múltiplos algoritmos de busca para melhorar a acurácia e a relevância dos resultados. A ideia central é usar o melhor dos dois mundos: busca por palavras-chave (sparse vectors com BM25) e busca semântica (dense vectors com modelos de embedding). Ao combinar essas abordagens, você obtém um sistema de busca muito mais robusto.

A arquitetura funciona assim: primeiro, você realiza duas buscas em paralelo – uma com sparse vectors usando BM25 e outra com dense vectors. Cada busca retorna uma lista ranqueada dos documentos mais similares, com seus respectivos scores. Depois, essas duas listas precisam ser combinadas em uma única lista final, removendo duplicatas. É aqui que entra o algoritmo de fusão.

No Qdrant, isso é implementado usando o parâmetro **prefetch**, que permite definir múltiplas estratégias de busca que serão executadas antes da combinação final. Cada estratégia é configurada com seu próprio tipo de vetor (dense ou sparse), seu próprio limite de candidatos, e o tipo de vetor a ser usado. Toda essa orquestração acontece no servidor, sem necessidade de combinar resultados manualmente no código Python. Na prática, o score subiu de 60% (busca densa pura) para 66% com hybrid search usando RRF – uma melhoria real, mas que ainda não é o teto.

> *"Hybrid search is a technique that combines multiple search algorithms to improve the accuracy and relevance of the results. The idea is to use the best of both worlds, keyword-based search and semantic vector search."*

Ao projetar um sistema de retrieval, comece sempre com hybrid

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

search. A implementação é simples com ferramentas como o Qdrant (que suporta nativamente sparse e dense vectors no mesmo documento), e o ganho de relevância em relação à busca densa pura é consistente e mensurável.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Reciprocal Rank Fusion resolve o problema de combinar rankings de escalas diferentes

Quando você tem duas listas ranqueadas de documentos – uma do BM25 e outra da busca vetorial – como combiná-las de forma justa? Os scores do BM25 podem ser valores como 15 ou 20, enquanto os da busca vetorial ficam sempre entre 0 e 1. São escalas completamente diferentes. Se você tentasse somar os scores diretamente, o resultado não funcionaria bem. É aqui que o **Reciprocal Rank Fusion (RRF)** entra com uma solução elegante.

A ideia do RRF é simples: ele ignora completamente os valores dos scores e trabalha apenas com as posições dos documentos nas listas. Para cada documento, o RRF calcula um score somando o inverso da posição em cada lista. Se o documento A aparece em primeiro no BM25 ($1/1 = 1.0$) e em terceiro na busca vetorial ($1/3 = 0.33$), seu score final é 1.33. Se o documento B aparece em primeiro na busca vetorial ($1/1 = 1.0$) e em segundo no BM25 ($1/2 = 0.5$), seu score final é 1.5.

O insight central é que documentos bem posicionados em ambas as listas acabam com scores altos, mesmo que não sejam os primeiros em nenhuma delas. No exemplo do curso, o documento B venceu com 1.5 sem ser primeiro em nenhuma lista individual – ele era consistentemente bem ranqueado nas duas. Essa propriedade torna o RRF extremamente robusto, porque não depende da magnitude dos scores originais, apenas da ordem relativa.

> *"The big insight of RRF is that it only looks at the position of the documents, not their original scores. This is important because BM25 can return scores like 15 or 20, while vector search always*

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

> *stays between 0 and 1. They're totally different scales."*

O RRF é um algoritmo simples de implementar e surpreendentemente eficaz. Sempre que você precisar combinar resultados de fontes diferentes – não apenas BM25 e busca vetorial, mas qualquer par de rankings – considere o RRF como primeira opção. Ele funciona bem justamente porque é agnóstico em relação às escalas dos scores.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Re-ranking com ColBERT refina resultados usando interação token a token

Enquanto o RRF combina listas de resultados olhando apenas para posições, o re-ranking vai além: ele pega os resultados iniciais e os reordena usando processamento mais profundo do conteúdo dos documentos. Nesse estágio, entram modelos neurais como cross-encoders e modelos de late interaction como o ColBERT.

Cross-encoders são extremamente precisos, mas também extremamente lentos. Eles pegam a query e cada documento candidato, concatenam os dois e alimentam tudo junto no modelo para calcular um score de relevância. Como cada par query-documento precisa ser processado separadamente, isso se torna inviável com milhões de documentos. Por isso, cross-encoders só são usados como estágio final de re-ranking em um conjunto pequeno de candidatos.

O **ColBERT (Contextualized Late Interaction over BERT)** oferece uma alternativa mais eficiente. Em vez de gerar um único embedding para o documento inteiro, ele gera um embedding separado para cada token do texto. Se o documento tem 100 tokens, você tem 100 embeddings. A query também vira múltiplos embeddings. O score é calculado comparando cada token da query com todos os tokens do documento, pegando a maior similaridade para cada token da query, e somando tudo. Isso captura matches localizados – "Apple" pode ter match forte no início do documento, "financial risks" no meio, e "2024" no final. Na prática, o score saltou dos 66% do RRF para relevância máxima normalizada de 100%, com os três primeiros resultados acima de 88%.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

> *"Instead of generating a single embedding for the document, ColBERT generates a separate embedding for each token in the text. For each token in the query, it finds which token in the document has the highest similarity and stores that value. In the end, it sums up all these maximum similarities."*

Use ColBERT (ou outro modelo de re-ranking) como estágio final do seu pipeline, nunca como o primeiro. Ele é preciso demais e lento demais para ser aplicado a milhões de documentos. A estratégia correta é: filtre com hybrid search + RRF primeiro, depois re-ranqueie os top 20-100 candidatos com ColBERT.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

O Qdrant suporta múltiplos tipos de vetores no mesmo documento, e isso muda tudo

Um dos diferenciais práticos demonstrados no curso é que o Qdrant permite armazenar múltiplos tipos de vetores por documento: dense, sparse e até multi-vectors para ColBERT. Isso significa que cada chunk no banco vetorial carrega simultaneamente três representações diferentes do mesmo texto, e o sistema pode orquestrar buscas complexas combinando todas elas.

Na configuração da collection, isso se traduz em um dicionário de vetores onde cada chave representa uma estratégia: "dense" com 384 dimensões e cosine similarity, "sparse" com a configuração do BM25, e "colbert" com 128 dimensões por token e um MultivectorConfig com comparador MaxSim. Essa última configuração é o que permite ao Qdrant calcular a interação token a token do ColBERT diretamente no servidor.

O poder real aparece na Query API com **prefetch aninhado** (nested prefetch). O prefetch interno executa duas buscas em paralelo (dense e sparse), cada uma retornando 10 candidatos. O prefetch externo aplica RRF nesses resultados e seleciona os top 20. Por fim, a query final usa ColBERT para re-ranquear esses 20 candidatos e retornar os 3 mais relevantes. Tudo isso acontece em uma única chamada ao servidor – sem precisar combinar resultados manualmente no código Python.

> *"The cool thing about Qdrant is that it natively supports all of this. You can store multiple types of vectors per document, sparse, dense, even multi-vectors for ColBERT. And with Qdrant's query API, you can build complex search pipelines, combining prefetch, fusion, and re-ranking all on the server."*

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Ao escolher um vector database para produção, avalie se ele suporta múltiplos tipos de vetores por documento e orquestração de pipeline no servidor. Isso simplifica enormemente seu código e reduz a latência, porque toda a lógica de busca, fusão e reranking acontece em uma única chamada de rede.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Semantic chunking agrupa parágrafos por significado, não por posição no texto

A forma como você divide um documento em chunks é uma das decisões mais impactantes em um pipeline de RAG. A abordagem mais simples – dividir por parágrafos ou por número fixo de tokens – é rápida mas ignora completamente o significado do conteúdo. Parágrafos sobre o mesmo tema podem acabar em chunks separados, enquanto parágrafos sobre temas distintos ficam juntos simplesmente porque são adjacentes no texto.

O semantic chunking resolve isso usando embeddings e clustering. Primeiro, cada parágrafo é transformado em um embedding. Depois, o algoritmo HDBSCAN agrupa os parágrafos cujos embeddings são próximos no espaço vetorial – ou seja, parágrafos que falam sobre o mesmo assunto. O resultado são chunks semanticamente coerentes, onde cada chunk contém parágrafos tematicamente relacionados, independentemente de onde apareciam no documento original.

Além do agrupamento, o semantic chunker implementado no curso respeita um limite máximo de tokens por chunk. Dentro de cada cluster, os parágrafos são acumulados até que o limite seja atingido. Quando um parágrafo novo estouraria o limite, o chunk atual é finalizado e um novo começa. Isso garante que nenhum chunk fique grande demais para o modelo de embedding processar adequadamente, mantendo ao mesmo tempo a coerência semântica.

> *"Chunking is one of the most important steps in the entire RAG pipeline. And there's no magic to it. The quality of your results fundamentally depends on how much you experiment and test dif-*

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

> *ferent approaches."*

Não existe uma estratégia de chunking universal. O semantic chunking funciona bem para documentos complexos que misturam vários temas (como relatórios financeiros), mas pode ser excessivo para documentos bem estruturados com seções claras. Sempre teste e compare diferentes abordagens no seu domínio específico.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

HDBSCAN encontra clusters automaticamente sem você precisar definir quantos existem

O algoritmo de clustering usado no semantic chunking é o HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise). A grande vantagem sobre algoritmos como k-means é que você não precisa definir previamente quantos clusters quer – o HDBSCAN descobre isso automaticamente, analisando a densidade dos pontos no espaço vetorial.

O funcionamento é intuitivo: se três ou mais parágrafos têm embeddings muito próximos entre si, eles formam um cluster. Se um parágrafo não está próximo de nenhum outro, ele é marcado como "ruído" – ou, na terminologia do curso, um "órfão". O parâmetro min_cluster_size define quantos pontos, no mínimo, precisam estar juntos para formar um cluster válido. E a métrica Euclidean é usada em vez de cosine distance porque o HDBSCAN precisa da noção de densidade espacial real – não apenas direção, mas também magnitude dos vetores.

O tratamento dos órfãos é igualmente sofisticado. Em vez de simplesmente descartá-los ou transformá-los em chunks individuais, o sistema tenta reagrupá-los com um critério menos restritivo (orphan_cluster_size menor que min_cluster_size). Os que conseguem se agrupar formam novos chunks. Os que permanecem completamente isolados se tornam chunks individuais. Essa abordagem em camadas maximiza a qualidade semântica dos chunks sem perder conteúdo.

> *"HDBSCAN stands for Hierarchical Density Based Spatial Clustering of Applications with Noise. The difference between HDBSCAN and other clustering algorithms like k-means is that you*

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

> *don't need to specify how many clusters you want. It figures that out automatically based on the density of the points."*

Ao usar HDBSCAN para semantic chunking, preste atenção especial aos parâmetros min_cluster_size e à métrica de distância. Use Euclidean (não cosine) porque o HDBSCAN depende da densidade espacial dos pontos. E sempre trate os órfãos com uma segunda rodada de clustering mais permissiva antes de transformá-los em chunks individuais.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Metadados estruturados transformam seu banco vetorial em uma base de conhecimento filtrável

Um banco vetorial sem metadados é como um arquivo cheio de documentos sem etiquetas – você pode buscar por conteúdo, mas não pode filtrar por contexto. No curso, a integração com a API do EDGAR (o sistema da SEC americana) demonstra como metadados bem estruturados elevam drasticamente a utilidade do sistema de busca.

Cada chunk inserido no Qdrant carrega não apenas seus vetores (dense, sparse e ColBERT), mas também um payload de metadados contendo: o ticker da empresa (AAPL, MSFT, NVDA), o nome da empresa, a data do relatório, e o tipo de formulário (10-K para relatórios anuais, 10-Q para trimestrais). Essa estrutura permite queries poderosas como "me mostre os riscos financeiros da Apple no relatório anual de 2024" – onde a busca vetorial encontra chunks sobre riscos financeiros e os metadados filtram por empresa e período.

A separação entre 10-K e 10-Q também importa por motivos semânticos. O relatório anual contém informações auditadas e abrangentes, enquanto o trimestral traz dados mais recentes mas menos detalhados. Ao manter cada chunk associado ao seu tipo de formulário, o sistema consegue distinguir entre análises de longo prazo e snapshots trimestrais. Além disso, a seleção cuidadosa dos itens extraídos de cada formulário (como item 1A para riscos, item 7 para análise da gestão) evita que ruído desnecessário polua a base de dados.

> *"This metadata will be very important later when we're making queries in Qdrant, because we'll be able to filter by company, by*

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

> *date, or by report type."*

Sempre projete seu schema de metadados antes de começar a ingestão. Pense em quais filtros seus usuários vão precisar: empresa, data, tipo de documento, seção, idioma. Metadados bem planejados tornam a diferença entre um sistema que "busca" e um sistema que "responde" com precisão.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Resumo Final

A ciência por trás da busca vetorial não se resume a gerar embeddings e calcular cosine similarity. O curso de Daniel Romero demonstra, de forma incremental e prática, que um sistema de retrieval de alta qualidade é um pipeline com múltiplas camadas, cada uma compensando as limitações da anterior.

Mensagem Central: Busca vetorial eficaz não é sobre um único algoritmo perfeito – é sobre a orquestração inteligente de múltiplas estratégias que, juntas, entregam resultados muito superiores ao que qualquer uma delas conseguiria sozinha.

O pipeline completo segue uma hierarquia clara:

1. **Ingestão inteligente** – semantic chunking com HDBSCAN para criar chunks semanticamente coerentes, respeitando limites de tokens e tratando órfãos com critério.
2. **Múltiplas representações** – cada chunk armazena três tipos de vetores (dense, sparse e ColBERT), cobrindo tanto o significado semântico quanto a correspondência exata de termos.
3. **Metadados estruturados** – cada chunk carrega informações de contexto (empresa, data, tipo de relatório) que permitem filtragem precisa.
4. **Busca em camadas** – hybrid search com dense e sparse vectors em paralelo, fusão com RRF para combinar rankings de escalas diferentes, e re-ranking final com ColBERT para precisão token a token.

A evolução dos scores ao longo do curso conta a história: de 60% com busca densa pura, para 66% com hybrid search + RRF, até relevância máxima normalizada com re-ranking via ColBERT.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Cada camada adicionou valor mensurável.

Conselho prático

Comece simples e evolua incrementalmente. Implemente primeiro a busca densa básica e meça o score. Adicione sparse vectors e hybrid search com RRF – meça o score de novo. Depois, adicione re-ranking com ColBERT. Em cada etapa, compare os resultados quantitativamente. Essa abordagem incremental não só facilita o debugging como também te mostra exatamente onde cada técnica agrega valor no seu domínio específico. E lembre-se: chunking e metadados importam tanto quanto o algoritmo de busca.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Sobre este resumo

Este resumo foi gerado a partir do curso **"A Ciência por Trás da Busca Vetorial"** de **Daniel Romero**.

O curso aborda a construção incremental de um sistema de retrieval para documentos financeiros, cobrindo desde dense embeddings e busca básica até hybrid search com sparse vectors (BM25), fusão com Reciprocal Rank Fusion (RRF), re-ranking com ColBERT, semantic chunking com HDBSCAN e integração com a API do EDGAR (SEC). O projeto prático utiliza Python, FastEmbed, Qdrant e a biblioteca edgartools.

Este blink foi criado para oferecer uma visão rápida e prática dos principais conceitos do curso. Para aprofundamento com exercícios e código, recomendamos o curso completo.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174